cli-multipurpose - Development Plan

Interactive ethical-hacking helper | Hybrid backends with automatic fallback

1. Goal

A CLI tool that lets the user select a security/recon tool from an interactive menu, answer a few prompts, and get results - instead of typing long command lines by hand. Every module ships TWO backends behind one interface: a native Python implementation (zero external downloads, works today) and an optional wrapper around a real CLI tool (nmap, sqlmap, gobuster, ...) used automatically when that tool is installed, e.g. on WSL/Kali.

2. Architecture

- Each module exposes: MENU_LABEL (menu text), PROMPTS (list of questions to ask), and run(target, options).
- run() validates input, then dispatches: if the real tool is available, run_tool(); otherwise run_native().
- Results are tagged with the backend used: [native] or [nmap].
- The tool works zero-install today and silently upgrades to full power on systems with the real tools.
- main.py only knows the module interface - it never touches tool details. Adding a tool = adding one module.

3. Module Matrix

Module	Native backend	Tool backend (when present)
http_headers	urllib HTTP request	curl -sIL
dns_scanner	Cloudflare DoH API	dig / nslookup
whois	RDAP (RFC 7480)	whois
subdomain_enum	crt.sh certificate log	subfinder -d
port_scanner	socket + concurrent.futures threaded TCP scan	nmap -sV
web_dir_scan	requests wordlist brute-force	gobuster dir
sql_detector	basic error-based payload check	sqlmap --batch

4. File Layout

Path	Purpose
main.py	Interactive menu loop, registry-driven
modules/__init__.py	REGISTRY dict: name -> module
modules/http_headers.py	HTTP response headers
modules/dns_scanner.py	DNS record lookup
modules/whois.py	Domain registration info
modules/subdomain_enum.py	Subdomain discovery
modules/port_scanner.py	TCP port scanning
modules/web_dir_scan.py	Directory/file brute-force
modules/sql_detector.py	Basic SQLi detection
utils/runner.py	tool_exists / run / first_available (subprocess)
utils/validation.py	validate_target / validate_ports
utils/formatter.py	section / ok / warn / err / table + ANSI colors

utils/net.py Shared HTTP / DoH / RDAP / crt.sh helpers

5. Build Order

1. Skeleton - __init__.py files; 'python -c "import modules, utils"' passes.
2. utils/runner.py - tool_exists (shutil.which + cache), run(cmd_list) with arg-list only and NO shell=True, utf-8 decode, exit-code + timeout handling. Test: run(['python','--version']).
3. utils/validation.py - validate_target (ipaddress, else domain regex), validate_ports (1-65535). Test good + malicious inputs.
4. utils/formatter.py - section, ok/warn/err/info, table; ANSI codes (needs Windows Terminal).
5. utils/net.py - urllib helpers: get_json(url) with timeout + error wrapping.
6. modules/http_headers.py - full dual-backend module; testable NOW (urllib always works, curl exists here). Template for all later modules.
7. modules/dns_scanner.py - DoH native (testable now), dig/nslookup fallback.
8. modules/whois.py - RDAP native (testable now), whois fallback.
9. modules/subdomain_enum.py - crt.sh native (testable now), subfinder fallback.
10. main.py - registry-driven menu: availability pass, generic PROMPTS rendering, target-confirmation gate, global try/except, 'q' to quit. 4 working modules by now.
11. port_scanner.py - socket threaded TCP scan native, nmap -sV backend.
12. web_dir_scan.py - requests brute-force native, gobuster dir backend.
13. sql_detector.py - error-based detector native, sqlmap backend.
14. Polish - result tagging, optional --dry-run, README + tool install checklist.

6. Key Design Decisions

- One module interface (MENU_LABEL / PROMPTS / run) is what makes the menu trivial and additions trivial.
- PROMPTS metadata drives the menu generically - adding an option later is editing a list, not menu code.
- Arg-list subprocess, never shell=True - the injection defense, and cross-platform for free.
- Respect free APIs (crt.sh / DoH): keep requests modest, always set timeouts.
- Port scanner is the deliberate weak spot - socket scan reports open/closed only; nmap -sV appears whenever nmap exists.

7. Verification Strategy

- Modules http_headers / dns_scanner / whois / subdomain_enum are fully testable on a stock Windows box today.
- port_scanner / web_dir_scan / sql_detector test the native path locally; the tool path is verified on WSL/Kali.
- Missing-tool path is a valid test by itself: graceful '[missing: nmap]' messages, never a crash.
- Only dependency for the zero-install core: pip install requests (and even that is optional - urllib covers everything).